FETCH System Administrator Installation Guide

Audience: System administrators installing FETCH (Findings Environment for Collected Holdings) for the first time from the open-source repositories.

Covers: Architecture, prerequisites, database requirements, a local evaluation install with Docker Compose, a production install on Kubernetes, Single Sign-On (SAML) setup, first-admin bootstrap, configuration reference, operations, and troubleshooting.

Conventions used in this guide

Placeholder	Meaning	Example
`fetch.example.org`	Public hostname of the FETCH web application	`fetch.mylibrary.edu`
`inventory.fetch.example.org`	Public hostname of the Inventory Service API	`inventory.fetch.mylibrary.edu`
`registry.example.org/fetch`	Your container registry / project path	`ghcr.io/mylibrary/fetch`
`admin@example.org`	Email address of the first FETCH administrator	`jdoe@mylibrary.edu`
`<...>`	A value you must supply	

> ***Note:*** *The web app and the API must share the same parent domain (for example* ***`fetch.example.org`*** *and* ***`inventory.fetch.example.org`****). FETCH authenticates with a browser cookie that is only sent between hosts on the same site. See Section 7.*

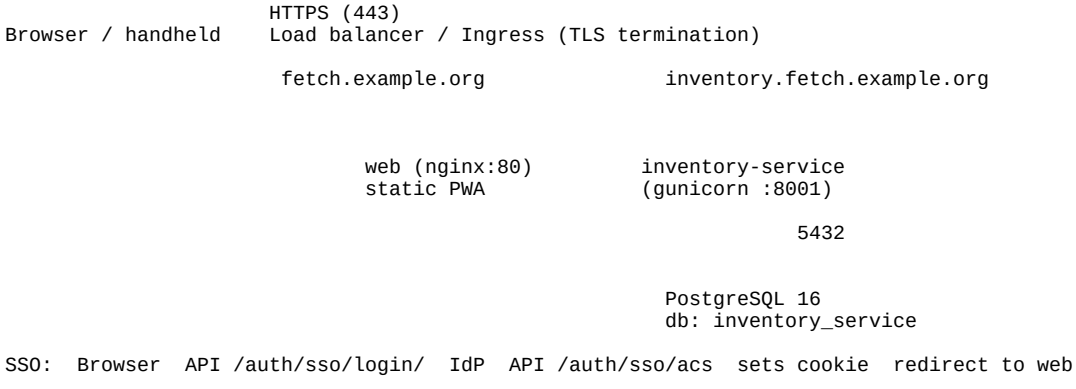
---

1. Architecture Overview

1.1 Components

Component	Repository	Technology	Listens on	Purpose
Web application	`fetch-vue`	Vue 3 + Quasar (Progressive Web App), served by nginx	Port 80 in the container	The user interface staff use on desktops and handheld scanners
Inventory Service API	`fetch-inventory_service`	Python 3.11, FastAPI, SQLAlchemy, Alembic; served by gunicorn/uvicorn	Port 8001 in the container	All business logic, authentication, background jobs, database migrations
Database	`fetch-database` (images)	PostgreSQL 16	5432	Stores all FETCH data in a database named `inventory_service`
Local developer stack	`fetch-fetch-local`	Docker Compose	n/a	Builds and runs the stack on a single workstation
Cluster build tooling	`fetch-build`	Ansible, HAProxy configs, monitoring compose files	n/a	Reference scripts the Library of Congress uses to build its Kubernetes hosts
Test automation (optional)	`fetch-automation`	Java / Maven UI tests	n/a	Not required for installation
Identity Provider (yours)	n/a	Any SAML 2.0 IdP (e.g., Microsoft Entra ID, Okta, Shibboleth)	n/a	Production login
Integrated Library System (optional, yours)	n/a	FOLIO, Alma, or custom middleware	n/a	Optional item-status sync, configured in the FETCH Admin UI after install

1.2 Request flow



The web application is a static bundle. Every data call goes from the browser directly to the API hostname. The web container never talks to the API or the database itself.

1.3 What the API does automatically at startup

Each time the Inventory Service starts, it:

1. **Runs database migrations** (`alembic upgrade head`). On an empty database this creates the entire schema from a single baseline migration, so no manual schema scripts are needed.
2. **Seeds the permission list.** It inserts any missing permissions (about 67) from `app/seed/fixtures/types/client_permissions.json`. This is safe to repeat.
3. **Starts a background scheduler** for scheduled report exports and the nightly cleanup of expired exports.

It does **not** create any users, groups, buildings, or reference data. Section 5.6 and Section 6.10 explain how to create the first administrator.

1.4 Repository layout expected by the tooling

The local Compose file builds each service from a sibling folder using fixed names. Clone the repositories into one parent folder using **these folder names**:

GitHub repository	Clone into folder
`LibraryOfCongress/fetch-fetch-local`	`fetch-local`
`LibraryOfCongress/fetch-database`	`database`
`LibraryOfCongress/fetch-inventory_service`	`inventory_service`
`LibraryOfCongress/fetch-vue`	`vue`
`LibraryOfCongress/fetch-build`	`build` (optional)

---

2. Choose an Installation Path

	Path A: Local evaluation (Docker Compose)	Path B: Production (Kubernetes)
Use for	Trying FETCH, training, development, testing configuration	Real operations
Hosts	One workstation or VM	Kubernetes cluster plus a PostgreSQL server
Login	Email-only login (no password, no SSO)	SAML Single Sign-On
TLS	None (plain HTTP on `localhost`)	Required (HTTPS at the load balancer / ingress)
Time to first login	About 30–60 minutes	About half a day, plus IdP registration lead time
Section	Section 5	Section 6

> ***Warning:*** *Never expose a Path A (local/test) installation to a network. In the `local`, `debug`, `develop`, and `test` environment modes, the API accepts requests without a login and lets anyone log in by typing an email address.*

---

3. Prerequisites

3.1 Software

Item	Path A	Path B	Notes
Git	Required	Required	To clone the repositories
Docker Desktop, Docker Engine + Compose v2, or Podman + podman-compose	Required	Required on the build machine	Compose commands in this guide use `docker compose`; substitute `podman compose` if you use Podman
Kubernetes cluster (1.27+) and `kubectl`	n/a	Required	The reference deployment runs Kubernetes 1.31 (kubeadm + containerd on RHEL-family Linux)
Ingress controller or external load balancer	n/a	Required	Examples use ingress-nginx; any L7 proxy that terminates TLS works
Container registry	n/a	Required	You build your own images; no pre-built images are published

PostgreSQL 16 server	Provided by Compose	Required	See Section 4
SAML 2.0 Identity Provider	n/a	Required	See Section 7
TLS certificates	n/a	Required	For both public hostnames, or a wildcard
`openssl`	Recommended	Required	To generate the JWT signing secret

3.2 Hardware sizing

Role	Minimum	Recommended (production)	Basis
Local evaluation workstation	4 CPU, 8 GB RAM allocated to Docker, 20 GB disk	8 GB RAM, 200 GB disk if loading real data	The Compose database settings assume about 8 GB is available
Inventory Service API (per pod)	1 CPU, 1 GiB RAM	Request 1 CPU / 1 GiB; limit 2 CPU / 4 GiB	Reference production deployment (5 gunicorn workers)
Web application (per pod)	0.25 CPU, 256 MiB RAM	Request 250m / 256Mi; limit 500m / 512Mi; 2 replicas	Reference production deployment
PostgreSQL server	4 CPU, 16 GB RAM, SSD storage	8 CPU, 32 GB RAM for collections of roughly 10 million items	Engine settings in Section 4.3; the repo's search-optimization guide recommends 32 GB at 10M items

3.3 Decisions to make before you start (Path B)

Collect these before starting a production install:

1. **Hostnames** for the web app and the API, both under the same parent domain.
2. **TLS certificates** for both hostnames.
3. **Database hosting**: an external or managed PostgreSQL 16 server (recommended), or PostgreSQL inside the cluster.
4. **Container registry** location and pull credentials.
5. **IdP registration**: who in your organization can register a SAML application, and how long that takes.
6. **First administrator's email address.** It must exactly match the email your IdP sends for that person.
7. **Where secrets live**: Kubernetes Secrets, Vault, or a sealed-secrets/external-secrets tool.

---

4. Database Requirements

4.1 Engine and database

Requirement	Value
Engine	PostgreSQL **16.x** (the schema baseline was produced on 16.8; the Compose and in-cluster images use `postgres:16.8`)
Database name	`inventory_service` (keep this name; helper scripts and documentation assume it)
Encoding	UTF8
Extensions	`pgcrypto` (the first migration runs `CREATE EXTENSION IF NOT EXISTS pgcrypto`)
Schema	`public`
Schema management	Automatic. The API runs Alembic migrations at startup. Do **not** load schema by hand.
Initial data	None. Permissions are seeded by the API; everything else is entered through the Admin UI (Section 9).

4.2 Create the database and application role (Path B)

Run as a PostgreSQL superuser on your database server:

```
CREATE ROLE fetch_app WITH LOGIN PASSWORD '<strong-password>';
CREATE DATABASE inventory_service OWNER fetch_app ENCODING 'UTF8';
\c inventory_service
-- pgcrypto is a "trusted" extension, so the database owner can create it.
-- Creating it here up front avoids any privilege surprises during the first migration:
CREATE EXTENSION IF NOT EXISTS pgcrypto WITH SCHEMA public;
ALTER SCHEMA public OWNER TO fetch_app;
```

The connection string you give the API looks like this:

```
postgresql://fetch_app:<strong-password>@<db-host>:5432/inventory_service
```

> ***Note:*** *If the password contains `@`, `:`, `/` or `%`, URL-encode it (for example, `@` becomes `%40`).*

4.3 Recommended engine settings

These are the settings the reference production database uses, mirrored in `fetch-local/compose.yml`. Scale memory values to your server (typically `shared_buffers` 25% of RAM and `effective_cache_size` 50–75% of RAM).

Setting	Value	Setting	Value
`max_connections`	500	`work_mem`	48MB
`shared_buffers`	3GB	`effective_cache_size`	8GB
`maintenance_work_mem`	256MB	`checkpoint_completion_target`	0.9
`checkpoint_timeout`	10min	`wal_buffers`	16MB
`max_wal_size`	2GB	`wal_compression`	on
`statement_timeout`	300s	`idle_in_transaction_session_timeout`	35s
`deadlock_timeout`	1s	`default_statistics_target`	100
`max_worker_processes`	8	`max_parallel_workers`	8
`max_parallel_workers_per_gather`	2	`autovacuum_max_workers`	3
`autovacuum_naptime`	1min	`autovacuum_vacuum_scale_factor`	0.2

Connection budget: each gunicorn worker keeps a pool of up to 15 connections (5 pooled + 10 overflow). Bulk import and seeding jobs can use up to 40 more. With the default 5 workers, plan for **at least 100 connections per API replica**, plus room for administrators and backups.

4.4 Storage and backups

- **Storage:** use SSD-backed storage. Size it for your collection: allow several GB per million items, including indexes and WAL. The reference in-cluster example in the `fetch-database` repo uses a 1 GiB volume, which is only suitable for demonstrations.
- **Backups:** schedule regular `pg_dump` custom-format backups, and always take one before an upgrade:

```
pg_dump -h <db-host> -U fetch_app -Fc -f inventory_service_$(date +%F).dump inventory_service
```

- **Restore** into an empty database:

```
pg_restore -h <db-host> -U fetch_app -d inventory_service --no-owner --no-privileges inventory_service_<date>.dump
```

4.5 Optional: performance indexes for large collections

For collections in the millions of items, apply the extra indexes in `inventory_service/scripts/search_optimization.sql` after the first startup. See `inventory_service/SEARCH_OPTIMIZATION.md` for details.

```
psql -h <db-host> -U fetch_app -d inventory_service -f inventory_service/scripts/search_optimization.sql
```

---

5. Path A: Local Evaluation Install (Docker Compose)

This path runs four containers on one machine:

Container	URL
`fetch-web`	http://localhost:8000
`fetch-inventory-api`	http://localhost:8001 (interactive API docs at http://localhost:8001/docs)
`inventory-database`	`localhost:5432` (user `postgres`, password `postgres`)
`fetch-pgadmin4`	http://localhost:5050 (login `admin@fetch.example.com` / `admin`; see `fetch-local/compose.yml`)

5.1 Prepare the container runtime

- **Docker Desktop:** under Settings Resources, allocate at least **8 GB memory** (the database is configured with 3 GB shared buffers) and enough disk for images and data.
- Create the host log folder the API container mounts: `sudo mkdir -p /var/log/fetch`. If you prefer another location, change the `inventory-api volumes` entry in `compose.yml`.

5.2 Clone the repositories

```
mkdir -p ~/workspace/fetch && cd ~/workspace/fetch
git clone https://github.com/LibraryOfCongress/fetch-fetch-local.git      fetch-local
git clone https://github.com/LibraryOfCongress/fetch-database.git         database
git clone https://github.com/LibraryOfCongress/fetch-inventory_service.git inventory_service
git clone https://github.com/LibraryOfCongress/fetch-vue.git              vue
```

> ***Note:*** *The folder names on the right are required.* `fetch-local/compose.yml` *builds from* `../database`, `../inventory_service`, *and* `../vue`.

5.3 Create the API configuration file

Create `inventory_service/app/config/.env`. The local image copies this file in at build time.

```
cd ~/workspace/fetch/inventory_service
cp .env.example app/config/.env
```

Edit `app/config/.env` so it contains:

```
APP_ENVIRONMENT=local
JWT_SECRET_KEY=<output of: openssl rand -hex 32>
DATABASE_URL=postgresql://postgres:postgres@inventory-database:5432/inventory_service
MIGRATION_URL=postgresql://postgres:postgres@localhost:5432/inventory_service
VUE_HOST=http://localhost:8000
ALLOWED_ORIGINS=http://localhost:8000,http://127.0.0.1:8000
```

`JWT_SECRET_KEY` is mandatory: the API refuses to start without it.

5.4 Create the web application configuration file

The local web image builds with `ENVIRONMENT=local`, which reads `vue/env/.env` and then `vue/env/.env.local`.

```
cd ~/workspace/fetch/vue
cp env/.env.example env/.env.local
```

Edit `env/.env.local` so it contains:

```
VITE_ENV=local
VITE_BASE_URL=/
VITE_API_BASE_URI=http://localhost:8000
VITE_INV_SERVCE_API=http://localhost:8001
```

> ***Note:*** *`VITE_INV_SERVCE_API` must have* ***no trailing slash****. The variable name really is spelled `SERVCE`.*

> ***Note:*** *These values are compiled into the web bundle at build time. If you change them, rebuild the `web` image (`docker compose up -d --build web`).*

5.5 Build and start the stack

```
cd ~/workspace/fetch/fetch-local
docker compose up -d --build
```

The first build takes several minutes. Then check:

```
docker compose ps
docker logs fetch-inventory-api --tail 50
```

A healthy API log ends with lines like these:

```
Running Alembic migrations...
Migrations complete.
Checking for new permissions...
Permission check complete.
Background Scheduler started.
```

> ***If `fetch-inventory-api` exited:*** *on the very first start the API can come up before PostgreSQL is ready to accept connections. Start it again once the database is up: `docker compose up -d inventory-api`.*

Check the API:

```
curl http://localhost:8001/status/
# {"detail":"OK"}
```

5.6 Create the first administrator

The database starts with no users. Run the bootstrap command **after** the API has started successfully at least once, because the permissions it attaches are seeded at API startup:

```
docker exec -it fetch-inventory-api \
  python -m app.seed.run_bootstrap_admin admin@example.org Local Admin
```

This creates (or reuses) a group called **System Administrators**, attaches **every** permission to it, creates the user if needed, and adds the user to the group. It is safe to re-run.

5.7 Log in

1. Browse to **http://localhost:8000**. Use `localhost`, not `127.0.0.1`: the login cookie is tied to the hostname the API was called with.
2. Enter `admin@example.org` and select **Login**. In local mode there is no password check.
3. Open **Admin** from the menu and continue with Section 9.

5.8 Optional: load demonstration data

To explore FETCH with a sample building, aisles, ladders, shelves (about 2,400) and shelf positions (about 41,000), plus sample users and groups:

```
docker exec -it fetch-inventory-api python -m app.seed.run_seed_fake
```

This takes several minutes. The sample users are defined in `inventory_service/app/seed/fixtures/entities/users.json`. Use this only on a throwaway database.

5.9 Day-to-day commands

Task	Command (run in `fetch-local`)
Stop the stack	`docker compose stop`
Start the stack	`docker compose up -d`
Rebuild after pulling new code	`docker compose up -d --build`
API logs	`docker logs -f fetch-inventory-api`
psql shell	`docker exec -it -u postgres inventory-database psql inventory_service`
Wipe all data and start over	`docker compose down -v` then `docker compose up -d --build`, then repeat 5.6

---

6. Path B: Production Install (Kubernetes)

6.1 Reference architecture (how the Library of Congress runs FETCH)

For context, the reference deployment in the public repositories uses:

- **Kubernetes 1.31** built with kubeadm and containerd on RHEL-family VMs (one control-plane node, two workers). Host preparation playbooks are in `fetch-build/ansible/k8s/`.
- **NodePort services:** the API on node port **30100** and the web app on **30200**, in namespace **`fetch`**.
- **HAProxy** on a separate VM terminates TLS and routes by Host header: `inventory.<domain>` goes to 30100 and `<domain>` goes to 30200. It health-checks `GET /status/` with 300-second timeouts. The config is in `fetch-build/ansible/haproxy/files/`.
- **GitLab CI** builds images per environment (`images/*.<env>.Containerfile`), writes the `.env` from CI variables, and deploys with the manifests in `k8s/` or the Terraform in `terraform/`. The Terraform reads the environment from HashiCorp Vault.
- **PostgreSQL 16** on a dedicated database server.

You do **not** need to copy this exactly. The rest of this section uses portable Kubernetes resources: an Ingress instead of NodePort+HAProxy, and Secrets instead of Vault. Equivalent options are noted where they differ.

6.2 Configure SAML for your Identity Provider

Follow Section 7 to register FETCH with your IdP and prepare `prod_saml_config.json`. You need it in step 6.5.

6.3 Build the Inventory Service image

The production Containerfile copies a `.env` file from the repository root into the image at `/code/app/config/.env`. That file must exist at build time. **Recommended:** keep only non-secret values in it and inject secrets at runtime (step 6.5). Values from the pod environment take precedence over the file.

```
cd inventory_service
printf 'APP_ENVIRONMENT=production\n' > .env

docker build -f images/api.prod.Containerfile -t registry.example.org/fetch/inventory-service:<version> .
docker push registry.example.org/fetch/inventory-service:<version>
```

> ***Note:*** *The reference CI writes* ***all*** *settings, including secrets, into this* `.env` *and bakes them into the image. That works, but anyone who can pull the image can read the secrets. Runtime injection is preferred.*

> ***SAML request signing:*** *if your IdP requires signed requests, uncomment the* `xmlsec` *build dependencies and the matching* `pip install` *line in* `images/api.prod.Containerfile` *before building.*

6.4 Build the web application image

The web app's settings are **compiled into the bundle at build time**, so you build one image per environment (for example, staging and production).

```
cd vue
cat > env/.env <<'EOF'
VITE_ENV=production
VITE_BASE_URL=/
VITE_API_BASE_URI=https://fetch.example.org
VITE_INV_SERVCE_API=https://inventory.fetch.example.org
EOF

docker build -f images/web.prod.Containerfile -t registry.example.org/fetch/web:<version> .
docker push registry.example.org/fetch/web:<version>
```

- `VITE_ENV=production` (or `stage`) makes the login page show **SSO Login**. Any other value shows the email-only login.
- `VITE_INV_SERVCE_API` has **no trailing slash**.
- The `certificates/` folder is used only during the build stage. If your build network inspects TLS traffic, replace those files with your organization's CA bundle. Otherwise leave them as they are.
- The image's nginx (`nginx/production.conf`) serves the app on port 80; TLS is terminated at the Ingress. You may change `server_name` to your hostname.

6.5 Create the namespace and secrets

```
kubectl create namespace fetch

# Registry pull credentials
kubectl -n fetch create secret docker-registry fetch-registry \
  --docker-server=registry.example.org --docker-username=<user> --docker-password=<token>

# API runtime configuration (secrets + settings)
kubectl -n fetch create secret generic inventory-service-env \
  --from-literal=APP_ENVIRONMENT=production \
  --from-literal=JWT_SECRET_KEY="$(openssl rand -hex 32)" \
  --from-literal=DATABASE_URL='postgresql://fetch_app:<password>@<db-host>:5432/inventory_service' \
  --from-literal=MIGRATION_URL='postgresql://fetch_app:<password>@<db-host>:5432/inventory_service' \
  --from-literal=VUE_HOST='https://fetch.example.org' \
  --from-literal=ALLOWED_ORIGINS='https://fetch.example.org' \
  --from-literal=ALLOWED_ORIGINS_REGEX='^https://fetch\.example\.org$' \
  --from-literal=TIMEZONE='America/New_York'

# SAML settings file (prepared in Section 7)
kubectl -n fetch create configmap inventory-service-saml \
  --from-file=prod_saml_config.json=./prod_saml_config.json
```

Mounting the SAML file from a ConfigMap (step 6.7) means you can change IdP settings without rebuilding the image. Alternatively, edit `app/saml/config/prod_saml_config.json` in the repository before step 6.3 and skip the ConfigMap.

> ***Keep*** `JWT_SECRET_KEY` ***stable.*** *Changing it logs out all users.*

6.6 Provide the database

Option 1 (recommended): external or managed PostgreSQL. Prepare it as described in Section 4 and point `DATABASE_URL` at it. Make sure the cluster's worker nodes can reach the database server on port 5432.

Option 2: PostgreSQL inside the cluster. This mirrors the `fetch-database/terraform` module. Set `DATABASE_URL` to `postgresql://postgres:<password>@inventory-database:5432/inventory_service`.

```
apiVersion: v1
kind: Secret
metadata:
  name: inventory-database
  namespace: fetch
type: Opaque
stringData:
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: <strong-password>
  POSTGRES_DB: inventory_service
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: inventory-database
  namespace: fetch
spec:
  serviceName: inventory-database
  replicas: 1
  selector:
    matchLabels: { app: inventory-database }
  template:
    metadata:
      labels: { app: inventory-database }
    spec:
      containers:
        - name: postgres
          image: postgres:16.8
          envFrom:
            - secretRef: { name: inventory-database }
          env:
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          args: ["-c", "max_connections=500", "-c", "shared_buffers=3GB",
                 "-c", "effective_cache_size=8GB", "-c", "work_mem=48MB",
                 "-c", "statement_timeout=300s", "-c", "idle_in_transaction_session_timeout=35s"]
          ports:
            - containerPort: 5432
          resources:
            requests: { cpu: "2", memory: 8Gi }
            limits:   { cpu: "4", memory: 16Gi }
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
            - name: dshm
              mountPath: /dev/shm
      volumes:
        - name: dshm
          emptyDir: { medium: Memory, sizeLimit: 2Gi }
  volumeClaimTemplates:
    - metadata:
        name: postgres-storage
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: <your-storage-class>
        resources:
          requests:
            storage: 100Gi
---
apiVersion: v1
kind: Service
metadata:
  name: inventory-database
  namespace: fetch
spec:
  selector: { app: inventory-database }
  ports:
    - port: 5432
      targetPort: 5432
  type: ClusterIP
```

6.7 Deploy the Inventory Service

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: inventory-service
  namespace: fetch
spec:
  replicas: 1
  selector:
    matchLabels: { app: inventory-service }
  template:
    metadata:
      labels: { app: inventory-service }
    spec:
      imagePullSecrets:
        - name: fetch-registry
      containers:
        - name: inventory-service
          image: registry.example.org/fetch/inventory-service:<version>
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8001
          envFrom:
            - secretRef: { name: inventory-service-env }
          volumeMounts:
            - name: saml
              mountPath: /code/app/saml/config/prod_saml_config.json
              subPath: prod_saml_config.json
              readOnly: true
            - name: exports
              mountPath: /code/app/data/exports
          startupProbe:
            httpGet: { path: /status/, port: 8001 }
            periodSeconds: 10
            failureThreshold: 30
          readinessProbe:
            httpGet: { path: /status/, port: 8001 }
            periodSeconds: 15
          livenessProbe:
            httpGet: { path: /status/, port: 8001 }
            periodSeconds: 30
            timeoutSeconds: 10
          resources:
            requests: { cpu: "1000m", memory: 1024Mi, ephemeral-storage: 200Mi }
            limits:   { cpu: "2000m", memory: 4096Mi, ephemeral-storage: 500Mi }
      volumes:
        - name: saml
          configMap: { name: inventory-service-saml }
        - name: exports
          persistentVolumeClaim: { claimName: inventory-service-exports }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: inventory-service-exports
  namespace: fetch
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: <your-storage-class>
  resources:
    requests:
      storage: 10Gi
---
apiVersion: v1
kind: Service
metadata:
  name: inventory-service
  namespace: fetch
spec:
  selector: { app: inventory-service }
  ports:
    - port: 80
      targetPort: 8001
  type: ClusterIP
```

Notes:

- **Run one replica.** The API runs its background scheduler (scheduled exports) inside the pod and writes export files to `/code/app/data/exports`. The reference deployment runs a single replica with 5 gunicorn workers. To scale beyond one replica, the export volume must be shared (ReadWriteMany).
- **The exports volume is optional.** It keeps scheduled report files across pod restarts. Without it they are lost on restart and count against the pod's ephemeral-storage limit.

- **Changing worker count:** the image runs `gunicorn ... --workers 5`. A common rule is 2 × CPU cores + 1. To change it, add `args:` to the container that repeat the full gunicorn command with a different `--workers` value.
- **Reference alternative:** to match the reference setup instead of using an Ingress, make this Service `type: NodePort` with `nodePort: 30100`, and point your external load balancer at the worker nodes.

6.8 Run the first database migration

On an empty database the first migration builds the whole schema. Run it once as a Job before the API pods start serving, so the gunicorn workers don't all try to migrate at the same moment:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: inventory-service-migrate
  namespace: fetch
spec:
  backoffLimit: 2
  template:
    spec:
      restartPolicy: Never
      imagePullSecrets:
        - name: fetch-registry
      containers:
        - name: migrate
          image: registry.example.org/fetch/inventory-service:<version>
          workingDir: /code
          command: ["alembic", "upgrade", "head"]
          envFrom:
            - secretRef: { name: inventory-service-env }
```

```
kubectl apply -f migrate-job.yaml
kubectl -n fetch wait --for=condition=complete job/inventory-service-migrate --timeout=300s
kubectl -n fetch logs job/inventory-service-migrate
kubectl -n fetch delete job inventory-service-migrate
```

Then apply the Deployment from 6.7 and wait for it:

```
kubectl apply -f inventory-service.yaml
kubectl -n fetch rollout status deployment/inventory-service --timeout=300s
kubectl -n fetch logs deployment/inventory-service --tail=50
```

Look for `Migrations complete.`, `Permission check complete.` and `Background Scheduler started.` in the logs.

6.9 Deploy the web application and Ingress

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vue
  namespace: fetch
spec:
  replicas: 2
  selector:
    matchLabels: { app: vue }
  template:
    metadata:
      labels: { app: vue }
    spec:
      imagePullSecrets:
        - name: fetch-registry
      containers:
        - name: vue
          image: registry.example.org/fetch/web:<version>
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet: { path: /, port: 80 }
          resources:
            requests: { cpu: 250m, memory: 256Mi, ephemeral-storage: 50Mi }
            limits:   { cpu: 500m, memory: 512Mi, ephemeral-storage: 200Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: vue
  namespace: fetch
spec:
  selector: { app: vue }
  ports:
    - port: 80
      targetPort: 80
```

```
  type: ClusterIP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: fetch
  namespace: fetch
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "50m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "300"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "300"
spec:
  ingressClassName: nginx
  tls:
    - hosts: [fetch.example.org, inventory.fetch.example.org]
      secretName: fetch-tls
  rules:
    - host: fetch.example.org
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: vue, port: { number: 80 } }
    - host: inventory.fetch.example.org
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: inventory-service, port: { number: 80 } }
```

Create the TLS secret from your certificate and key, then apply:

```
kubectl -n fetch create secret tls fetch-tls --cert=fetch.crt --key=fetch.key
kubectl apply -f web-and-ingress.yaml
```

Requirements for any proxy or load balancer in front of FETCH:

- Terminate TLS and forward to the services over HTTP.
- **Preserve the original `Host` header.** If the proxy sets `X-Forwarded-Host` or `X-Forwarded-Port`, they must carry the public values (`inventory.fetch.example.org`, 443). SAML validation compares these against the configured ACS URL.
- Allow request bodies large enough for batch uploads (50 MB is a safe start).
- Use read timeouts of at least 300 seconds. Some reports and bulk operations are long-running.

Verify:

```
curl https://inventory.fetch.example.org/status/        # {"detail":"OK"}
curl -I https://fetch.example.org/                        # HTTP 200
curl https://inventory.fetch.example.org/auth/sso/metadata # SP metadata XML
```

6.10 Create the first administrator

Users are created automatically the first time they sign in with SSO, but **with no groups, and therefore no access**. Grant the first administrator full access with the bootstrap command:

```
kubectl -n fetch exec -it deploy/inventory-service -- \
  python -m app.seed.run_bootstrap_admin admin@example.org First Last
```

- Run it after the API has started (so permissions exist). It can be run before or after the person's first SSO login.
- The email must **exactly** match the email claim your IdP sends.
- The command is safe to re-run. Re-running it also adds any newly introduced permissions to the System Administrators group.

The administrator can now browse to `https://fetch.example.org`, select **SSO Login**, and continue with Section 9. If you cannot `exec` into pods, see Appendix A for an SQL alternative.

6.11 Upgrading

1. Take a database backup (Section 4.4).
2. Pull the new code, then rebuild and push both images with a new tag (6.3, 6.4).
3. Run the migration Job (6.8) with the new API image tag.
4. Update the image tags in the Deployments and apply them. Wait for `rollout status`.
5. New permissions are added automatically at startup but are **not** assigned to any group. Re-run `run_bootstrap_admin` to refresh the System Administrators group, and assign new permissions to other groups in **Admin Groups & Permissions**.

---

7. Configuring Single Sign-On (SAML 2.0)

7.1 How login works

1. The user selects **SSO Login**. The browser goes to `https://inventory.fetch.example.org/auth/sso/login/`.
2. The API redirects the browser to your IdP.
3. The IdP posts the SAML response to the API's Assertion Consumer Service (ACS): `https://inventory.fetch.example.org/auth/sso/acs`.
4. The API finds the user by email or creates them, then sets an **HttpOnly, Secure, SameSite=Lax** cookie named `fetch_auth_token`. The session lasts 15 minutes and is extended on activity.
5. The API redirects the browser to `VUE_HOST`, where the web app loads the user's profile from `/auth/me`.

Because the cookie is set by the API host and sent with the web app's API calls, **both hostnames must be on the same site** (same registrable domain) and served over **HTTPS**.

7.2 Register FETCH with your IdP

Create a SAML 2.0 application ("Enterprise application" in Entra ID) with these Service Provider values:

IdP field	Value
Entity ID / Identifier / Audience	`https://inventory.fetch.example.org/` (include the trailing slash, and match the JSON file exactly)
ACS / Reply URL	`https://inventory.fetch.example.org/auth/sso/acs` (HTTP-POST binding)
Sign-on URL (optional)	`https://fetch.example.org/`
Logout URL	`https://inventory.fetch.example.org/auth/sso/logout`
NameID format	Email address (recommended)

FETCH reads three attributes from the SAML response, **using these exact attribute names**. They are the Microsoft Entra ID defaults. For other IdPs, add attribute mappings that release these names:

Attribute name (must match exactly)	Value
`http://schemas.xmlsoap.org/ws/2005/05/identity/claims/emailaddress`	User's email
`http://schemas.xmlsoap.org/ws/2005/05/identity/claims/givenname`	First name
`http://schemas.xmlsoap.org/ws/2005/05/identity/claims/surname`	Last name

Ask your IdP team for: the **IdP Entity ID** (Issuer), the **SSO URL** (HTTP-Redirect binding), the **Logout URL**, and the **signing certificate** (Base64, without the BEGIN/END lines).

7.3 Prepare the SAML settings file

The API chooses its SAML settings file from `APP_ENVIRONMENT`:

APP_ENVIRONMENT	File read (inside inventory_service/app/saml/config/)
`production`	`prod_saml_config.json`
`stage`	`stage_saml_config.json`
`test`	`test_saml_config.json`
`develop`	`dev_saml_config.json`
`local`, `debug`	`local_saml_config.json`

Start from `app/saml/config/prod_saml_config.json` and fill in your values:

```
{
  "strict": true,
  "debug": false,
  "security": { "requestedAuthnContext": false },
  "sp": {
    "entityId": "https://inventory.fetch.example.org/",
    "assertionConsumerService": {
      "url": "https://inventory.fetch.example.org/auth/sso/acs",
      "binding": "urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"
    },
    "singleLogoutService": {
      "url": "https://inventory.fetch.example.org/auth/sso/logout",
      "binding": "urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect"
    },
```

```
    "x509cert": "",
    "privateKey": ""
  },
  "idp": {
    "entityId": "<IdP Entity ID / Issuer>",
    "singleSignOnService": {
      "url": "<IdP SSO URL>",
      "binding": "urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect"
    },
    "singleLogoutService": {
      "url": "<IdP Logout URL>",
      "binding": "urn:oasis:names:tc:SAML:2.0:bindings:HTTP-Redirect"
    },
    "x509cert": "<IdP signing certificate, Base64, single line>"
  }
}
```

Use this file in step 6.5 (ConfigMap) or commit it to your fork before building the image.

> ***Note:*** *The `IDP_ENTITY_ID` and `IDP_LOGIN_URL` environment variables exist in the settings class but are **not** used for login. The JSON file is the source of truth.*

7.4 Verify SSO

1. `https://inventory.fetch.example.org/auth/sso/metadata` should return SP metadata XML. Many IdPs can import this directly.
2. Sign in from `https://fetch.example.org`. After the IdP round trip you should land back in FETCH.
3. If you sign in but see no menus, the user has no groups. Run the bootstrap command (6.10) or have an administrator add the user to a group.

---

8. Configuration Reference

8.1 Inventory Service (API) settings

Set these as environment variables (preferred) or in `app/config/.env`. Environment variables take precedence over the file.

Variable	Required	Default	Description
`APP_ENVIRONMENT`	Yes	`local`	One of `production`, `stage`, `test`, `develop`, `local`, `debug`. Controls security behavior (see 8.2) and which SAML file is read.
`JWT_SECRET_KEY`	**Yes**	none	Secret for signing session tokens. Generate with `openssl rand -hex 32`. The API will not start without it.
`DATABASE_URL`	Yes	placeholder	`postgresql://user:password@host:5432/inventory_service`
`MIGRATION_URL`	No	placeholder	Database URL used only when running `alembic` from outside a container with `USE_MIGRATION_URL=true`. Set it equal to `DATABASE_URL` in deployed environments.
`VUE_HOST`	Yes (SSO)	`https://localhost:8000`	Public URL of the web app. The API redirects here after SSO. No trailing slash.
`ALLOWED_ORIGINS`	Yes	localhost list	Comma-separated list of exact origins allowed to call the API with credentials, e.g. `https://fetch.example.org`
`ALLOWED_ORIGINS_REGEX`	Recommended	example pattern	A **single** regular expression for allowed origins, e.g. `^https://fetch\.example\.org$`
`TIMEZONE`	No	`America/New_York`	Facility time zone (IANA name)
`APP_NAME`	No	`FETCH`	Display name returned by the API root
`ENABLE_ORM_SQL_LOGGING`	No	`false`	Logs every SQL statement. Troubleshooting only.
`FORWARDED_ALLOW_IPS`	No	`*` (set in the production image)	Which proxies are trusted to send `X-Forwarded-*` headers

8.2 What `APP_ENVIRONMENT` changes

Behavior	`production` / `stage`	`test`	`develop`	`local` / `debug`
Email-only login (`/auth/legacy/login`)	Disabled	Enabled	Enabled	Enabled
API calls without a login, or with an expired/invalid session	Rejected (401)	**Allowed**	**Allowed**	**Allowed**
Cookie `Secure` flag	On (HTTPS only)	On	On	Off
Forces `https://` on redirects	Yes	Yes	Yes	No

> ***Warning:*** *Use `production` (or `stage`) for every environment reachable by other people.*

8.3 Web application settings (build time)

Set in `vue/env/.env` (deployed builds) or `vue/env/.env.local` (local Compose build). Rebuild the image after any change.

Variable	Example	Description
`VITE_ENV`	`production`	`production` or `stage` shows **SSO Login** and uses the faster scanner input timing. Any other value shows the email-only login form. `production` also hides the internal test page.
`VITE_INV_SERVCE_API`	`https://inventory.fetch.example.org`	Public URL of the API. No trailing slash.
`VITE_BASE_URL`	`/`	Path the app is served under
`VITE_API_BASE_URI`	`https://fetch.example.org`	Public URL of the web app

Keep the two sides consistent:

Scenario	API `APP_ENVIRONMENT`	Web `VITE_ENV`
Production with SSO	`production`	`production`
Staging with SSO	`stage`	`stage`
Isolated test (email login)	`test`	`test`
Local Compose	`local`	`local`

A mismatch is the most common setup mistake. For example, a web build with `VITE_ENV=test` against an API with `APP_ENVIRONMENT=production` shows an email login form that the API rejects.

8.4 Where configuration lives in the repositories

File	Purpose
`inventory_service/.env.example`	Template for API settings
`inventory_service/app/config/config.py`	Definitive list of API settings and defaults
`inventory_service/app/saml/config/*_saml_config.json`	SAML settings per environment
`inventory_service/images/api.<env>.Containerfile`	API image definitions (`local`, `dev`, `test`, `stage`, `prod`)
`inventory_service/k8s/`, `inventory_service/terraform/`	Reference API deployment manifests
`vue/env/.env.example`	Template for web settings
`vue/images/web.<env>.Containerfile`	Web image definitions
`vue/nginx/*.conf`	nginx configuration per environment
`vue/k8s/`, `vue/terraform/`	Reference web deployment manifests
`fetch-local/compose.yml`	Local stack definition, including database tuning
`database/inventory/images/*.Containerfile`	PostgreSQL image definitions
`database/terraform/`	Reference in-cluster PostgreSQL StatefulSet
`build/ansible/`	Reference host preparation (Kubernetes nodes, HAProxy)

---

9. First-Time Application Setup

After the first administrator can log in, configure FETCH from the **Admin** dashboard in this order. `inventory_service/CONFIGURATION.md` describes each screen in detail.

1. **Groups & Permissions:** create groups that match your roles (e.g. Accessioners, Shelving, Supervisors, Administrators), assign permissions, and add users. Users appear after their first SSO login. They can also be pre-created with `run_bootstrap_admin` (admin access) or added through **User Management**.
2. **Owners:** your ownership hierarchy (owner tiers and parent/child owners).
3. **Media Types**
4. **Size Classes** (width, depth, height in inches)
5. **Shelf Types**, linked to size classes
6. **Barcode Types:** regex patterns for item, tray, and shelf barcodes
7. **Priorities**
8. **Delivery Locations**
9. **Request Types**
10. **Location Manager:** Building Module Aisle Side Ladder Shelf Shelf Position. **Bulk Create Shelves/Ladder** accepts a CSV for large facilities.
11. **System Configurations:** shelf position direction, child-owner shelving, shipping module
12. **ILS Integrations** (optional): FOLIO, Alma, or custom middleware. See `inventory_service/ILS_INTEGRATION.md`. Treat ILS client secrets as credentials.

> ***Migrating from a legacy system?*** *The `inventory_service/migration/` folder has ETL guides (`ETL_DATA_MIGRATION_GUIDE.md`, `DATA_MAPPING_MATRIX.md`). Plan that work separately from the base install.*

---

10. Operations

Task	How
Health check	`GET https://inventory.fetch.example.org/status/` returns `{"detail":"OK"}`
API logs	Written to stdout: `kubectl -n fetch logs deploy/inventory-service` (Path A: `docker logs fetch-inventory-api`)
API docs	`https://inventory.fetch.example.org/docs` (interactive OpenAPI)
Restart the API	`kubectl -n fetch rollout restart deployment/inventory-service`
Grant full admin to a user	`kubectl -n fetch exec -it deploy/inventory-service -- python -m app.seed. run_bootstrap_admin <email>`
Check the migration version	`kubectl -n fetch exec -it deploy/inventory-service -- alembic current`
Back up / restore	Section 4.4
Audit trail	Database changes are recorded by triggers in `audit_log` and viewable in the Admin UI
Monitoring (optional)	`build/docker/prometheus-grafana/` has a reference Prometheus/Grafana compose setup

---

11. Troubleshooting

Symptom	Likely cause	Fix
API exits with `JWT_SECRET_KEY Field required`	Secret not provided	Add `JWT_SECRET_KEY` to the Secret (Path B) or `app/config/.env` (Path A), then rebuild (Path A) or restart.
API exits on first `docker compose up` with a connection error	PostgreSQL was not ready yet	`docker compose up -d inventory-api`
`permission denied to create extension "pgcrypto"`	The app role can't create the extension	Run `CREATE EXTENSION pgcrypto;` as a superuser (Section 4.2)
`password authentication failed` or `could not translate host name`	Wrong `DATABASE_URL`	Check host, port, credentials, and URL-encoding of the password. From the pod: `psql "$DATABASE_URL" -c 'select 1'`
Web app loads but every action fails; requests go to `undefined/...`	Web built without `env/.env` values	Create `env/.env` (or `.env.local` for local) and rebuild the web image
Browser console shows CORS errors	Web origin not allowed	Add the exact web origin (scheme + host, no trailing slash) to `ALLOWED_ORIGINS` and restart the API
SSO returns to FETCH but you are still logged out, or every call returns 401	Session cookie not sent	Both hosts must be HTTPS and share a parent domain. Locally, browse to `localhost`, not `127.0.0.1`. Check `ALLOWED_ORIGINS`.
SSO error 400 mentioning destination, audience, or invalid response	SP values in the JSON don't match the IdP, or the proxy changed the	Make the entity ID and ACS URL match exactly in both places. Make sure the proxy preserves `Host` and sends public `X-`

	Host or port	`Forwarded-*` values.
`No matching saml config for <env> environment`	Unsupported `APP_ENVIRONMENT` value	Use one of the values in Section 8.2
SSO error `KeyError` on an `http://schemas.xmlsoap.org/...` name	IdP not releasing the required attributes	Add the attribute mappings in Section 7.2
User signs in but sees no menus or gets 403s	User has no groups	Run `run_bootstrap_admin` or add them to a group in Admin
`404` on `/auth/legacy/login`	Expected in `production`/`stage` (email login is disabled)	Build the web image with `VITE_ENV=production` so it uses SSO
Some API URLs return 404 when the trailing slash is wrong	The API doesn't redirect on slash mismatches (`redirect_slashes=False`)	Use the paths exactly as documented (e.g. `/status/`, `/auth/sso/login/`)
PWA "install" option missing or service worker not registering	Not served over HTTPS	Serve the web app over HTTPS (`localhost` is exempt)

Appendix A: SQL Alternative for the First Administrator

Use this only if you cannot run `run_bootstrap_admin` (for example, if you can't `exec` into pods). Run it after the API has started at least once, so the permissions exist. Replace the email and names.

```
BEGIN;

INSERT INTO groups (name, create_dt, update_dt)
VALUES ('System Administrators', now(), now())
ON CONFLICT (name) DO NOTHING;

INSERT INTO group_permissions (group_id, permission_id)
SELECT g.id, p.id
FROM groups g CROSS JOIN permissions p
WHERE g.name = 'System Administrators'
ON CONFLICT ON CONSTRAINT uq_permission_id_group_id DO NOTHING;

INSERT INTO users (first_name, last_name, email, create_dt, update_dt)
SELECT 'First', 'Last', 'admin@example.org', now(), now()
WHERE NOT EXISTS (SELECT 1 FROM users WHERE lower(email) = lower('admin@example.org'));

INSERT INTO user_groups (user_id, group_id)
SELECT u.id, g.id
FROM users u, groups g
WHERE lower(u.email) = lower('admin@example.org') AND g.name = 'System Administrators'
ON CONFLICT ON CONSTRAINT uq_user_id_group_id DO NOTHING;

COMMIT;
```

Check it worked:

```
SELECT u.email, g.name, count(gp.permission_id) AS permissions
FROM users u
JOIN user_groups ug ON ug.user_id = u.id
JOIN groups g ON g.id = ug.group_id
LEFT JOIN group_permissions gp ON gp.group_id = g.id
WHERE lower(u.email) = lower('admin@example.org')
GROUP BY u.email, g.name;
```

Appendix B: Installation Checklist

Local evaluation (Path A)

- Docker/Podman installed, with 8 GB memory allocated
- Repositories cloned into `fetch-local`, `database`, `inventory_service`, vue
- `inventory_service/app/config/.env` created with `JWT_SECRET_KEY`
- `vue/env/.env.local` created
- `docker compose up -d --build` completed; `/status/` returns OK
- `run_bootstrap_admin` run; logged in at http://localhost:8000

Production (Path B)

- Hostnames chosen (same parent domain), DNS and TLS certificates ready
- PostgreSQL 16 database `inventory_service`, app role, and `pgcrypto` created; backups scheduled
- IdP application registered; attributes mapped; `prod_saml_config.json` prepared
- API image built and pushed (`APP_ENVIRONMENT=production`)

- Web image built and pushed (`VITE_ENV=production`, API URL with no trailing slash)
- Namespace, pull secret, `inventory-service-env` Secret, and SAML ConfigMap created
- Migration Job completed
- API Deployment healthy; `/status/` OK; `/auth/sso/metadata` returns XML
- Web Deployment and Ingress applied; site loads over HTTPS
- `run_bootstrap_admin` run for the first administrator; SSO login works
- First-time application setup (Section 9) completed